



Tuxhydro

Análisis de la influencia de los índices oceánicos en los aportes hídricos en los embalses en Colombia.

Tuxito Lovers

Universidad Jorge Tadeo Lozano

Alvarado Becerra Ludwig

www.ludwigalvarado.me

Jiménez Quintero Nicolle Tatiana

tatiana.jima@gmail.com

Martínez Guerrero Juan José

info@cusvide.club

Ojeda Mejía María Sofía

sofiaojedamejia2004@gmail.com

Resumen

Tuxilo! [18]

Índice general

Resumen	i
Índice general	ii
Índice de figuras	v
1 Descripción general de la solución	1
2 Análisis técnico y metodología	3
2.1 Correlación	3
2.2 Predicción	3
2.2.1 Precipitación	3
2.2.2 Caudales	3
2.3 Implementación	6
3 Uso de datos y fuentes	7
3.1 Datos geoespaciales	7
3.2 Información hidrológica	8
3.2.1 Caudales	8
3.2.2 Precipitación	10
4 Herramientas, requerimientos de software y arquitectura	12
4.1 Componentes de la solución	12
4.2 Flujo general	15
4.2.1 Interfaz de visualización geográfica	15
4.2.2 Flujo de consulta y análisis	15
4.2.3 Interpretación de resultados	15
4.2.4 Consulta de datos mediante API	16
4.3 Configuraciones a ajustar	16
5 Código y replicabilidad	18
5.1 Frontend	18
5.1.1 Estructura del Frontend	18
5.1.2 Configuración y Dependencias	18
5.1.3 Scripts de Desarrollo	19
5.1.4 Conexión con el Backend	19

5.2	Backend	20
5.2.1	Estructura del Backend	20
5.2.1.1	Archivo Principal	20
5.2.1.2	Routers (APIs)	20
5.2.1.3	Modelos de Base de Datos	22
5.2.2	Dependencias del Backend	23
5.2.3	Ejecutar el Backend	23
5.2.4	Base de Datos	24
5.3	Notebooks de análisis	24
5.3.1	Ejecución con Marimo	24
5.4	Documentación	25
5.4.1	Estructura de la Documentación	26
5.4.2	Prerrequisitos para Compilar	26
5.4.3	Compilación de la Documentación	27
5.4.4	Proceso de Compilación Manual	27
5.4.5	Editar la Documentación	28
6	Resultados y visualización	29
7	Guía de despliegue e instalación operativa	30
7.1	Requerimientos técnicos	30
7.1.1	Requerimientos de hardware	30
7.1.2	Requerimientos de software	31
7.1.3	Sistemas operativos soportados	31
7.2	Instalación y configuración	31
7.2.1	Obtención del código fuente	32
7.2.2	Configuración del entorno	32
7.2.3	Configuración de secretos	32
7.2.4	Construcción de las imágenes	33
7.3	Despliegue y ejecución	34
7.3.1	Inicio de servicios	34
7.3.2	Verificación del despliegue	35
7.3.3	Acceso a la aplicación	36
7.3.4	Detención y reinicio de servicios	36
7.4	Recomendaciones para uso institucional	36
7.4.1	Seguridad y acceso	37
7.4.2	Monitoreo y mantenimiento	37
7.4.3	Actualización de datos	37
7.5	Escalabilidad y extensibilidad	38
7.5.1	Escalabilidad horizontal	38
7.5.2	Escalabilidad vertical	38
7.5.3	Extensiones funcionales	39
7.5.4	Migración a infraestructura en la nube	39
7.6	Solución de problemas comunes	39

7.6.1	Error al iniciar la base de datos	40
7.6.2	Frontend no carga correctamente	40
7.6.3	Datos no se cargan en la base de datos	40
7.6.4	Problemas de rendimiento	41

Bibliografía**42**

Índice de figuras

2.1	Flow direction y drenajes obtenidos por el DEM de la región.	5
3.1	Mapa elaborado con Leaflet utilizando datos base de OpenStreet- Map y capas vectoriales correspondientes a los embalses de Colom- bia.	8
3.2	Regiones que cubren a Colombia dentro del portal del GRDC.	9
3.3	Archivo GRDC/3102010_Q_Day.Cmd.txt	9
3.4	Archivo 21080030.csv.	11
4.1	Análisis de licencias de software utilizadas en el proyecto	14
5.1	Ejecución y edición de un notebook de marimo	25
7.1	Proceso terminado del build	34
7.2	Servicios corriendo en la maquina local	35

CAPÍTULO 1

Descripción general de la solución

El problema abordado en este reto se centra en la alta sensibilidad del sistema hídrico colombiano frente a la variabilidad climática, particularmente aquella asociada a fenómenos oceánicos globales. En un país cuya matriz energética depende en gran medida de la generación hidroeléctrica, al representar aproximadamente el 70% de la producción de energía nacional[28], la ausencia de herramientas analíticas que integren de manera sistemática los índices oceánicos y el comportamiento hidrológico de los embalses limita la capacidad de anticipación, planificación y gestión eficiente del recurso hídrico y energético.

Ante este contexto, se propone el desarrollo de un modelo analítico y predictivo que permita evaluar la relación entre los principales índices oceánicos, como El Niño-Oscilación del Sur (ENSO), el Índice de Niño Oceánico (ONI), la Oscilación del Sur (SOI), la Oscilación del Atlántico Norte (NAO), entre otros, y los aportes hídricos que reciben los embalses en Colombia. El modelo integra series históricas de caudales y niveles de embalse, con el fin de identificar correlaciones clave que permitan determinar su impacto sobre el sistema hídrico nacional.

El enfoque de la solución se basa en un aplicativo que permite estimar el comportamiento futuro de los embalses, determinando las variaciones en sus niveles a partir de la precipitación influenciada por los índices oceánicos y su efecto sobre los caudales de los ríos tributarios. De esta manera, el modelo ofrece servicios de análisis estadístico y exploratorio de datos hidrológicos y climáticos, que permiten caracterizar la influencia directa de los índices oceánicos sobre la variabilidad hidrológica nacional. A partir de este análisis, se incorporan herramientas predictivas básicas orientadas a estimar el comportamiento futuro de los aportes hídricos en función de la evolución de los índices oceánicos, fortaleciendo así la capacidad de anticipación frente a eventos de sequía o exceso hídrico.

La funcionalidad básica del modelo consiste en:

- La integración y procesamiento de datos históricos hidrológicos y climáticos;
- El análisis de correlación y temporalidad entre los índices oceánicos y los aportes hídricos;
- La generación de insumos analíticos y visuales que apoyen la toma de decisiones.

Esta solución es operable, coherente con los objetivos del reto y escalable, ya que puede implementarse como un modelo analítico, un tablero de visualización o una herramienta de apoyo a la toma de decisiones para el sector minero-energético. Su aplicación contribuye al fortalecimiento de la planificación operativa de los embalses, a la reducción de riesgos asociados a la variabilidad climática y al entendimiento de los efectos del cambio climático sobre el sistema hídrico colombiano, promoviendo decisiones basadas en evidencia y disminuyendo los riesgos operativos asociados a sequías o excesos hídricos.

CAPÍTULO 2

Análisis técnico y metodología

2.1 Correlación

2.2 Predicción

El modelo predictivo se estructura a partir de la integración de índices climáticos de gran escala que presentan una correlación significativa con las distintas regiones del país, identificada a partir del análisis desarrollado en el apartado anterior. Estos índices se emplean como variables explicativas para la estimación de la precipitación proyectada y, posteriormente, para el cálculo del caudal asociado a cada embalse.

Dicho modelo se aplica para la predicción del periodo comprendido entre 2025-1S y 2029-2S y se desarrolla en dos etapas consecutivas. En primer lugar, se modela la precipitación futura a partir de la relación estadística entre los índices climáticos y los registros históricos de precipitación. En una segunda etapa, los valores de precipitación proyectada se incorporan en un modelo empírico lluvia–escorrentía, con el fin de estimar el caudal esperado en cada embalse. Finalmente, los caudales proyectados se analizan en comparación con la tendencia histórica, lo que permite identificar posibles escenarios de incremento o disminución del volumen almacenado en los embalses.

A continuación, se describen de manera detallada cada una de las etapas que componen el modelo predictivo.

2.2.1 Precipitación

2.2.2 Caudales

Con base en los resultados obtenidos en la predicción de la precipitación, se estima el caudal de manera empírica a partir de relaciones lluvia–escorrentía, utilizando datos históricos de caudal y precipitación mensuales provenientes de estaciones hidrometeorológicas cercanas a cada embalse quienes se encuentran en el repositorio como `data/Calculo_caudal/Caudal_`

`Consolidado_Completo.csv`. Este enfoque se fundamenta en la premisa de que la precipitación constituye el principal aporte al caudal superficial y que su relación con la escorrentía puede representarse mediante expresiones matemáticas simplificadas, especialmente en contextos donde la disponibilidad de información detallada del balance hídrico es limitada[41].

En este sentido, se emplea un modelo empírico de tipo potencia, evidenciado en el repositorio como `Tuxilo/notebooks/CORRECTED_CAUDAL_CALCULATION.py`, que relaciona el caudal con la precipitación mediante coeficientes de ajuste, los cuales integran las características hidrológicas de la cuenca, tales como su tamaño y su respuesta ante eventos de lluvia, según la siguiente expresión:

$$Q_{i,t} = a_r \cdot (P_{i,t} + P_{i,t-1} + P_{i,t-2})^{b_r} \cdot \left(\frac{A_r}{A_i} \right) \quad (2.1)$$

Donde:

- $Q_{i,t}$ es el caudal estimado asociado al embalse i en el período t , expresado en m^3/s .
- a_r es el coeficiente empírico de escala, obtenido a partir del ajuste entre los datos históricos de precipitación y caudal, y representa la magnitud media de la respuesta hidrológica de la cuenca.
- b_r es el exponente de la relación lluvia-escorrentía, el cual refleja la sensibilidad del caudal ante variaciones en la precipitación y el comportamiento hidrológico de la cuenca.
- $P_{i,t}$ corresponde a la precipitación semestral proyectada en el área de influencia del embalse i durante el período t (mm).
- $P_{i,t-1}$ y $P_{i,t-2}$ representan la precipitación semestral de los dos meses anteriores, incorporadas para considerar el efecto acumulado y retardado de la precipitación sobre la generación de escorrentía.
- A_r es el área de la macrocuenca en la que se encuentra el embalse i (km^2).
- A_i es el área de drenaje del embalse, utilizada para normalizar el caudal estimado en función del tamaño relativo de la cuenca.
- t indica el período temporal de análisis (mes).
- i identifica cada uno de los embalses analizados.

Para la estimación de los coeficientes a_r y b_r , se aplicó una regresión lineal en escala logarítmica, considerando la precipitación acumulada en una ventana temporal de tres meses, código presente en `notebooks/improve_br_cumulative.py`, definida como:

$$P_{cum}(t) = P(t) + P(t-1) + P(t-2) \quad (2.2)$$

A partir de esta definición, la relación empírica entre caudal y precipitación se linealiza mediante la transformación logarítmica, expresándose como:

$$\ln(Q) = \ln(a_r) + b_r \cdot \ln(P_{\text{cum}}(t)) \quad (2.3)$$

donde los parámetros a_r y b_r se obtienen a partir del ajuste por mínimos cuadrados utilizando los datos históricos de precipitación y caudal.

Para la obtención de las áreas A_r y A_i , se utilizó la información geográfica de zonificación hidrográfica y el Modelo Digital de Elevación (DEM), descritas en el capítulo siguiente. El cálculo del área A_r se realizó en QGIS mediante la herramienta de cálculo de campos, aplicando la expresión $\text{area}/1000000$, posterior a la separación de las macrocuencas y a la aplicación de la herramienta *merge*.

Para la estimación del área A_i , se hizo uso de las herramientas del *toolbox* del aplicativo GRASS GIS, correspondientes a los procesos de análisis hidrológico sobre datos ráster y vectoriales [3]. En primer lugar, se empleó la función *r.fill.dir* para corregir depresiones y espacios en blanco presentes en el archivo ráster del DEM. Posteriormente, se utilizó la herramienta *r.watershed* para obtener la dirección de flujo y la red de drenaje de la cuenca a partir del relieve, considerando una escala mínima de análisis de 1000 unidades, como se muestra en la figura 2.1. Finalmente, se aplicó la herramienta *r.water.outlet*, en la cual se definió el punto de descarga correspondiente al embalse, permitiendo delimitar el área de influencia de drenaje asociada a cada uno, dando como resultado un archivo vectorial en formato *.shp*.

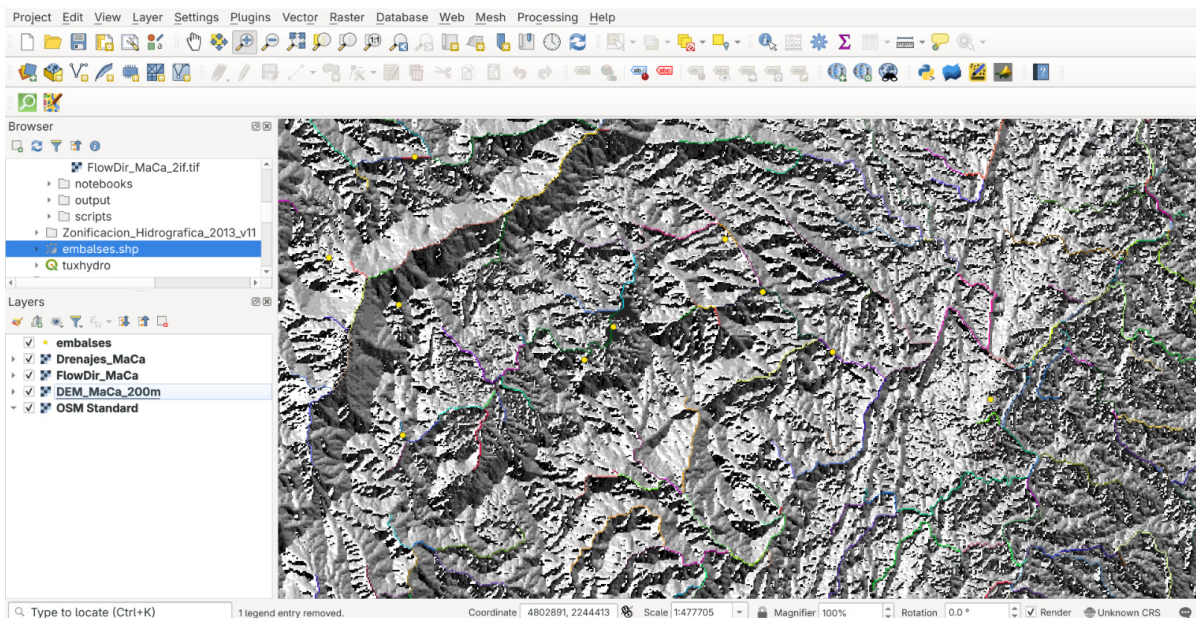


Figura 2.1: Flow direction y drenajes obtenidos por el DEM de la región.

Posterior a dicho cálculo, se realiza una comparación entre el caudal estimado para cada em-

balse y la tendencia de los caudales históricos, registrada en el repositorio ***. Esta comparación permite identificar los períodos en los que el caudal proyectado se sitúa por encima de la tendencia histórica, lo cual se asocia con una fase de aumento del aporte hídrico y, por ende, con una posible subida del nivel del embalse. En contraste, cuando el caudal proyectado se ubica por debajo de la línea de tendencia histórica, se interpreta como una disminución de los aportes, lo que sugiere una reducción del volumen almacenado en el embalse.

2.3 Implementación

CAPÍTULO 3

Uso de datos y fuentes

3.1 Datos geoespaciales

Para el desarrollo del proyecto se identificaron y utilizaron datos geoespaciales correspondientes a los polígonos asociados a los embalses y regiones hidrográficas a nivel nacional. Esta información fue obtenida a partir de la plataforma *Colombia en Mapas*, de los datos oficiales suministrados por el Instituto Geográfico Agustín Codazzi (IGAC)[21] y del portal del *IDEAM*[20]. Los insumos geográficos empleados se encuentran disponibles dentro del repositorio del proyecto, específicamente en el directorio `data/geodata/`. De igual forma, se utilizó el DEM asociado al shape de las zonas hidrográficas de Colombia, archivo `.tif` extraído con el plugin de QGIS *OpenTopography DEM Downloader* [33], con el fin de calcular los `.shp` de las cuencas de drenaje de cada embalse analizado, encontrado en el repositorio como `GIS/Cuencas_influencia_embalses`.

Los datos originales, proporcionados en formato `.shp`, fueron preparados para su integración en una base de datos espacial PostGIS[36]. Para ello, se realizó una conversión de las capas vectoriales a sentencias SQL compatibles con PostgreSQL, permitiendo la creación y carga estructurada de las tablas correspondientes. Este procedimiento asegura la correcta preservación de los atributos y geometrías, así como la incorporación de índices espaciales que optimizan las consultas posteriores. Dichas sentencias SQL se pueden encontrar en el archivo `init-embalses-geom-shp2pgsql.sql` del directorio `scripts/`.

Una vez almacenadas las geometrías de los embalses en la base de datos, se calcularon los centroides de cada polígono con el objetivo de obtener coordenadas representativas de ubicación. Estas coordenadas de latitud y longitud fueron utilizadas para poblar una tabla adicional que contiene la información general de cada embalse, facilitando su consulta desde otros componentes del sistema. Tanto la definición de las tablas como los procesos de inserción y cálculo geométrico se encuentran documentados y automatizados mediante scripts SQL incluidos en el repositorio en los archivos `init-db.sql` y `init-grdc-data.sql`.

Los conjuntos de datos generados son expuestos a través de una API REST, lo que permite su consumo por diferentes módulos del proyecto y su reutilización en futuros desarrollos. En particular, se dispone de un servicio para la consulta de la información general de los embalses en el endpoint `/public/embalses/` (identificador, nombre y coordenadas) y de otro servicio para la obtención de los polígonos en formato GeoJSON, adecuado para aplicaciones de visualización y análisis geoespacial (endpoint `public/embalses/geojson`). Estos servicios se ejecutan localmente y forman parte de la arquitectura de la aplicación.

Como capa base cartográfica se empleó la base de datos espacial de OpenStreetMap[2], haciendo uso de su información conforme a los términos establecidos por la Licencia Abierta de Bases de Datos (ODbL)[32]. Sobre esta capa base se integraron los datos propios del proyecto como capas vectoriales superpuestas, permitiendo la representación espacial de los embalses y sus atributos asociados.

Adicionalmente, se generó una capa dinámica en formato GeoJSON[49], obtenida mediante la consulta a la API, lo que posibilita la visualización directa de los polígonos de los embalses sin necesidad de almacenar archivos intermedios. La integración y representación gráfica de la información geográfica se realizó mediante la librería Leaflet[24] para JS, obteniendo como resultado una interfaz cartográfica interactiva, similar a la presentada en la figura 3.1. Esta aproximación permite una exploración visual eficiente de los datos y facilita su interpretación espacial.

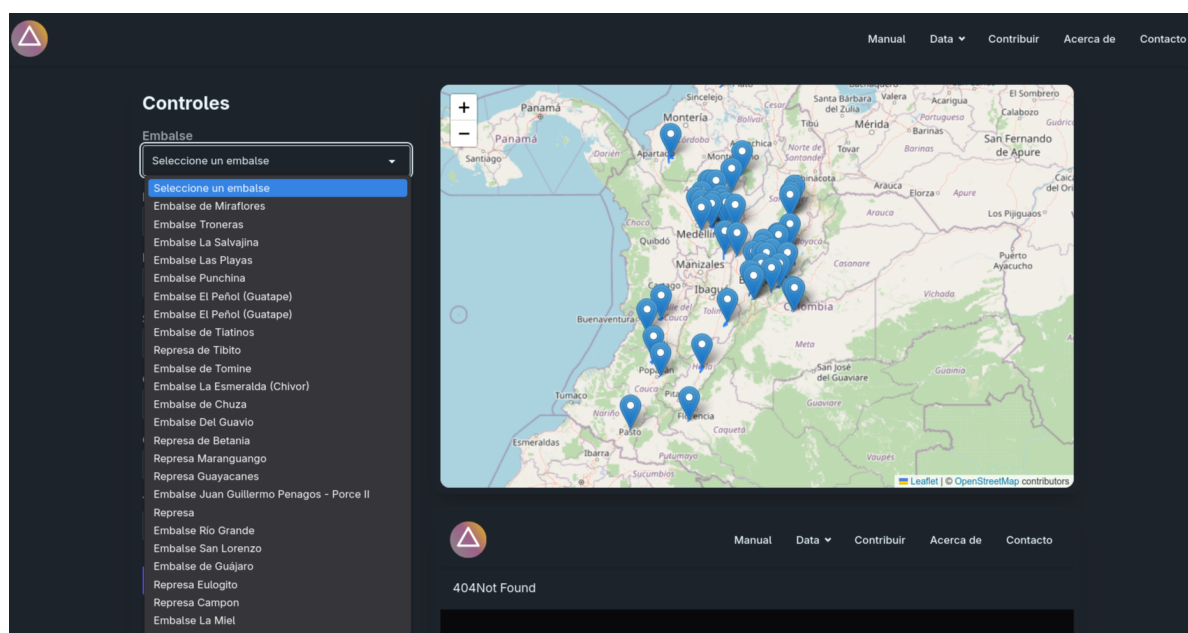


Figura 3.1: Mapa elaborado con Leaflet utilizando datos base de OpenStreet-Map y capas vectoriales correspondientes a los embalses de Colombia.

3.2 Información hidrológica

3.2.1 Caudales

Esta información se sacó del Global Runoff Data Centre (GRDC) (<https://portal.grdc.bafg.de/applications/public.html?publicuser=PublicUser#dataDownload/Subregions>)[17] seleccionando todas las regiones que cubren a Colombia (figura 3.2) generando un archivo comprimido enviado al correo del solicitante. Este archivo se descomprime y se pueden ver dentro de data/GRDC con un nombre tipo ID_Q_Day_Clean.Cmd.txt donde ID es el identificador de la estación meteorológica, un ejemplo de cómo vienen los datos están en la figura 3.3.

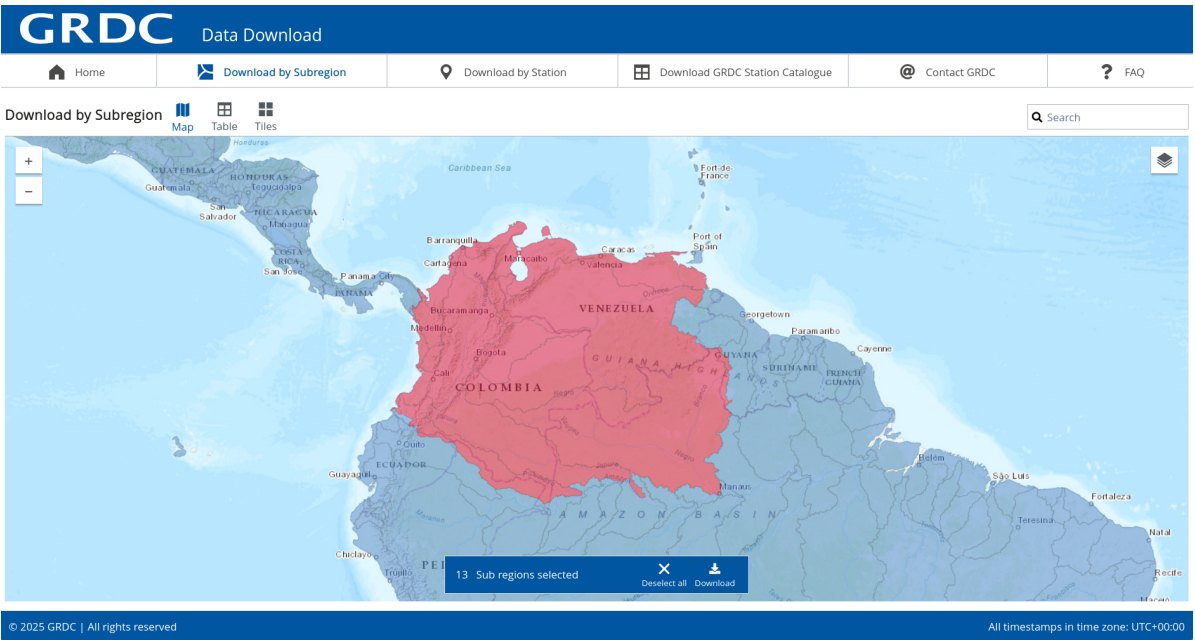


Figura 3.2: Regiones que cubren a Colombia dentro del portal del GRDC.

```
File: GRDC/3102010_Q_Day.Cmd.txt
1 # Title: GRDC STATION DATA FILE
2 # -----
3 # Format: DOS-ASCII
4 # Field delimiter: ;
5 # missing values are indicated by -999.000
6 #
7 # file generation date: 2025-12-18
8 #
9 # GRDC-No.: 3102010
10 # River: RIO SINU
11 # Station: COTOCA ABAJO - AUT [13077060]
12 # Country: CO
13 # Latitude (DD): 9.22444444
14 # Longitude (DD): -75.83444444
15 # Catchment area (km): 13397.3757
16 # Altitude (m ASL): 6.0
17 # Next downstream station: -
18 # Remarks:
19 # Owner of original data: Instituto de Hidrología, Meteorología y Estudios Ambientales
20 #*****
21 #
22 # Data Set Content: MEAN DAILY DISCHARGE (Q)
23 # -----
24 # Unit of measure: m/s
25 # Time series: 1970-03 - 2024-12
26 # No. of years: 55
27 # Last update: 2025-05-27
28 #
29 # Table Header:
30 # YYYY-MM-DD - Date
31 # hh:mm - Time
32 # Value - original (provided) data
33 #*****
34 #
35 # Data lines: 20030
36 # DATA
37 YYYY-MM-DD;hh:mm; Value
38 1970-03-01;--:--; 146.300
39 1970-03-02;--:--; 140.700
40 1970-03-03;--:--; 129.500
41 1970-03-04;--:--; 121.800
```

Figura 3.3: Archivo GRDC/3102010_Q_Day.Cmd.txt

Una vez realizada la descarga de los datos, se llevó a cabo un proceso de depuración y limpieza, debido a que una proporción significativa de las estaciones presentaba ausencia de registros de

caudal, series temporales desactualizadas o valores nulos. Con el fin de garantizar la consistencia y representatividad del conjunto de datos, se priorizaron aquellas estaciones que contaran con información a partir del año 2015.

Para el tratamiento de los valores faltantes, se aplicó una aproximación estadística basada en la distribución de los valores no nulos de cada serie. A partir de dicha distribución, se generaron muestras aleatorias (*sampling*) que permitieron reemplazar los valores nulos, preservando así las características estadísticas originales de los datos. El procedimiento completo de análisis y limpieza se encuentra documentado en el archivo `analisis_limpieza_caudal.py`, ubicado en el directorio `notebooks/`. Como resultado de este proceso, se obtuvieron los conjuntos de datos finales almacenados en el directorio `data/Caudal/`.

A partir del archivo `stationbasins.geojson`, disponible en el directorio `data/GRDC/`, se extrajeron las coordenadas geográficas correspondientes a cada estación meteorológica. Esta información fue procesada y consolidada en el archivo `caudal_stations.csv`, localizado en `data/Caudal/`, el cual contiene el identificador de la estación, su nombre, el río asociado y sus coordenadas de latitud y longitud. Dicho archivo en formato `.csv` se empleó posteriormente para la población de la tabla `stations` en la base de datos.

Por último, el archivo `caudal_flow.csv`, ubicado en el directorio `data/Caudal/`, corresponde a la integración de los registros diarios de caudal provenientes de todas las estaciones seleccionadas. Este conjunto de datos se utilizó para la carga y población de la tabla `grdc_daily_flow` en la base de datos.

3.2.2 Precipitación

La información utilizada en este estudio fue obtenida a partir del Instituto de Hidrología, Meteorología y Estudios Ambientales (IDEAM) [19], mediante la selección de estaciones meteorológicas cercanas a los embalses de interés. Los datos corresponden principalmente a registros de precipitación y se encuentran organizados en el repositorio `data/Precipitacion/data/originales`, los cuales se presentan de manera general en la Figura 3.4.

Para el análisis de la información, se realizó inicialmente un análisis exploratorio de los datos, con el objetivo de evaluar su calidad, continuidad y representatividad temporal. En esta etapa se identificaron estaciones con registros incompletos o con baja disponibilidad de datos, lo cual fue evidenciado a través de gráficas de tendencia y series temporales.

Adicionalmente, la información meteorológica fue contrastada con la ubicación espacial de los embalses, considerando su localización dentro de la macrocuenca hidrográfica de interés. Este proceso se realizó con base en información geoespacial de zonificación hidrográfica y localización de embalses, permitiendo establecer una relación coherente entre las estaciones seleccionadas, las áreas de drenaje y los sistemas de almacenamiento hídrico analizados.

File: 21080030.csv	
1	CodigoEstacion,NombreEstacion,Variable,Parametro,Fecha,Unidad,Valor,NivelAprobacion
2	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2015-01-01 00:00,mm,7,Definitivo
3	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2015-02-01 00:00,mm,34,Definitivo
4	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2015-03-01 00:00,mm,73,Definitivo
5	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2015-04-01 00:00,mm,31,Definitivo
6	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2015-05-01 00:00,mm,5,Definitivo
7	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2015-06-01 00:00,mm,14,Definitivo
8	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2015-07-01 00:00,mm,0,Definitivo
9	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2015-08-01 00:00,mm,0,Definitivo
10	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2015-09-01 00:00,mm,0,Definitivo
11	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2015-10-01 00:00,mm,35,Definitivo
12	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2015-11-01 00:00,mm,165,Definitivo
13	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2015-12-01 00:00,mm,0,Definitivo
14	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2016-01-01 00:00,mm,10,Definitivo
15	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2016-02-01 00:00,mm,20,Definitivo
16	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2016-03-01 00:00,mm,215,Definitivo
17	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2016-04-01 00:00,mm,170,Definitivo
18	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2016-05-01 00:00,mm,92,Definitivo
19	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2016-06-01 00:00,mm,20,Definitivo
20	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2016-07-01 00:00,mm,16,Definitivo
21	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2016-08-01 00:00,mm,15,Definitivo
22	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2016-09-01 00:00,mm,94,Definitivo
23	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2016-10-01 00:00,mm,100,Definitivo
24	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2016-11-01 00:00,mm,365,Definitivo
25	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2016-12-01 00:00,mm,317,Definitivo
26	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2017-01-01 00:00,mm,182,Definitivo
27	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2017-02-01 00:00,mm,101,Definitivo
28	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2017-03-01 00:00,mm,231,Definitivo
29	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2017-04-01 00:00,mm,141,Definitivo
30	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2017-05-01 00:00,mm,187,Definitivo
31	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2017-06-01 00:00,mm,23,Definitivo
32	21080030,YAGUARA [21080030],PRECIPITACION,Precipitación total mensual,2017-07-01 00:00,mm,15,Definitivo

Figura 3.4: Archivo 21080030.csv.

CAPÍTULO 4

Herramientas, requerimientos de software y arquitectura

4.1 Componentes de la solución

Para dar cumplimiento al requerimiento de uso de software libre, el cuadro 4.1 presenta el conjunto de programas empleados durante el desarrollo del proyecto, junto con una breve descripción de su uso y la licencia correspondiente. Para la clasificación de las licencias se adopta la definición de Free/Libre and Open Source Software (FLOSS), que incluye software libre y de código abierto, según lo establecido por la Free Software Foundation (FSF)[12].

El único software utilizado que no se enmarca dentro de la categoría FLOSS es GitHub, el cual fue empleado como plataforma de alojamiento del código fuente y como herramienta de colaboración entre los integrantes del equipo. No obstante, el proyecto no presenta *vendor lock-in*, ya que la dependencia de esta plataforma no es estructural. En consecuencia, es posible -y se recomienda- migrar el repositorio y los flujos de trabajo a una instancia propia de Gitea[16], lo que permitiría que el proyecto sea completamente desarrollado y gestionado con software libre y abierto.

Cuadro 4.1: Software utilizado, descripción y licencias

Software	Descripción	Licencia
Python	Lenguaje de programación usado durante todo el proyecto	PSFL (Open Source)[37]
Docker	Software de virtualización para ayudar al despliegue de todos los servicios	Apache 2.0 (Open Source)[29]
PostGIS	Modulo para soporte de objetos geográficos a PostgreSQL	GPL-2 (Libre)[36]
NGINX	Usado como servidor web dentro del proyecto	BSD-2 (Open Source)[30]

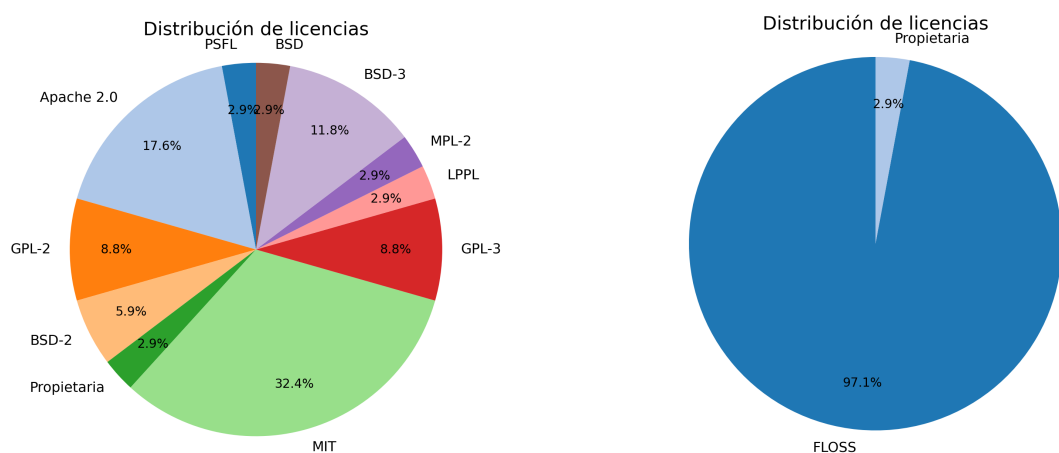
Continúa en la siguiente página

Software	Descripción	Licencia
GitHub	Servicio de hosting y colaboración para alojar el código	Propietario[50]
QGIS	Análisis de datos geoespaciales	GPL-2 (Libre)[38]
Marimo	Notebooks para trabajar, limpiar y procesar datos	Apache 2.0 (Open Source)[25]
Svelte	Framework de JS usado para el front-end del proyecto	MIT (Open Source)[43]
Leaflet	Librería de JS para mostrar y montar capas de datos geográficos teniendo como capa base OSM	BSD-2 (Open Source)[23]
FastAPI	Framework de Python utilizado para desarrollar las API del proyecto	MIT (Open Source)[8]
Drawio	Aplicación utilizada para dibujar la arquitectura	Apache 2.0 (Open Source)[6]
GNU Emacs	Editor de texto utilizado para editar archivos de datos, usado por sus macros	GPL-3 (Libre)[51]
L ^A T _E X	Sistema de software para la creación del presente documento	LPPL (Libre)[22]
Bash	Lenguaje de scripting usado para automatizar scripts	GPL-3 (Libre)[48]
GNU Make	Utilidad usada para compilar algunos archivos del proyecto	GPL-3 (Libre)[11]
LibreOffice Calc	Hojas de cálculo para realizar operaciones con datos	MPL-2 (Open Source)[4]
Git	Software de control de versiones distribuido para el desarrollo de todo el proyecto	GPL-2 (Libre)[15]
Aiofiles	Operaciones asincrónicas para leer y escribir archivos	Apache 2.0 (Open Source)[46]
Openpyxl	Librería de python para leer y escribir archivos de Excel	MIT (Open Source)[13]
Pandas	Manipular y analizar datos relacionales y etiquetados	BSD-3 (Open Source)[45]
Requests	Enviar solicitudes HTTP	Apache 2.0 (Open Source)[9]
SQLAlchemy	Conjunto de herramientas de SQL para Python	MIT (Open Source)[42]
GeoAlchemy2	Conjunto de herramientas para trabajar con bases de datos espaciales en Python	MIT (Open Source)[14]
Uvicorn	Servidor web ASGI para Python	BSD-3 (Open Source)[1]

Continúa en la siguiente página

Software	Descripción	Licencia
Plotly	Librería gráfica para Python	MIT (Open Source)[35]
PydataXM	Extraer información del mercado energético de Colombia	MIT (Open Source)[7]
Numpy	Librería para computación científica en Python	BSD-3 (Open Source)[31]
Matplotlib	Crear visualizaciones estáticas e interactivas en Python	BSD (Open Source)[26]
Seaborn	Librería de visualización	BSD-3 (Open Source)[40]
Bun	Conjunto de herramientas para JS y TS	MIT (Open Source)[34]
DaisyUI	Librería de componentes para Tailwind CSS	MIT (Open Source)[39]
Vite	Desarrollo frontend	MIT (Open Source)[47]
TypeScript	Sistema de tipado para ayudar a JS	Apache 2.0 (Open Source)[27]
TailwindCSS	Framework para construir UI	MIT (Open Source)[44]

En conclusión, la distribución de licencias FLOSS y propietaria se puede ver en la figura 4.1(b) donde se muestra que un 97.1% de las librerías y programas usados en el proyecto son FLOSS.



((a)) Distribución de licencias utilizadas en el proyecto

((b)) Distribución de licencias FLOSS y propietarias en el proyecto

Figura 4.1: Análisis de licencias de software utilizadas en el proyecto

4.2 Flujo general

4.2.1 Interfaz de visualización geográfica

La aplicación web principal presenta un mapa interactivo de Colombia que muestra la ubicación de los embalses analizados. La interfaz consta de los siguientes componentes:

- **Mapa base:** Utiliza capas de OpenStreetMap[2] con datos geospaciales cargados desde la API
- **Polígonos de embalses:** Representan el área de cada embalse, con popups informativos al hacer clic
- **Panel de controles:** Permite seleccionar un embalse y un período semestral de análisis
- **Gráficos de precipitación:** Muestra las series temporales de precipitación para la estación más cercana al embalse seleccionado

4.2.2 Flujo de consulta y análisis

El proceso típico de consulta sigue los siguientes pasos:

1. **Selección de embalse:** Utilizar el menú desplegable superior para elegir un embalse de interés
2. **Selección de período:** Elegir un semestre específico (por ejemplo, 2025-1S)
3. **Consultar:** Hacer clic en el botón "Consultar" para obtener la proyección de caudal
4. **Interpretación visual:** El sistema colorea el polígono del embalse según el resultado:
 - **Verde:** Indica aumento proyectado del caudal (por encima de la media histórica)
 - **Rojo:** Indica disminución proyectada del caudal (por debajo de la media histórica)
5. **Información detallada:** Al hacer clic sobre el polígono coloreado, se muestra un popup con el cambio exacto de caudal en m^3/s , indicando si se trata de un aumento o disminución respecto a la media

4.2.3 Interpretación de resultados

Los resultados de la proyección se basan en la relación histórica entre índices oceánicos, precipitación y caudales. La interpretación de los valores debe considerar lo siguiente:

- **Valores positivos de cambio:** Sugieren condiciones favorables para la generación hidroeléctrica, con mayor disponibilidad de recurso hídrico
- **Valores negativos de cambio:** Alertan sobre posibles escenarios de sequía o reducción del aporte hídrico
- **Magnitud del cambio:** La diferencia en m³/s permite cuantificar la severidad del escenario proyectado

Es importante considerar que estas proyecciones son estimaciones estadísticas basadas en patrones históricos y están sujetas a incertidumbre. Se recomienda utilizarlas como herramienta de apoyo a la toma de decisiones, complementándolas con otros análisis técnicos y pronósticos meteorológicos actualizados.

4.2.4 Consulta de datos mediante API

La API REST expuesta por el sistema permite acceder programáticamente a los datos. Los endpoints principales son:

- GET /public/embalses/points: Lista todos los embalses con sus coordenadas
- GET /public/embalses/polygons: Obtiene las geometrías de los embalses en formato GeoJSON
- GET /public/estaciones/flow: Lista las estaciones de medición de caudal
- GET /public/estaciones/rain: Lista las estaciones de medición de precipitación
- GET /public/data/semestres: Obtiene la lista de períodos semestrales disponibles
- GET /public/data/q_proyectado/{embalse_id}/{periodo}: Obtiene la proyección de caudal para un embalse y período específicos
- GET /public/data/media/{graph_file}: Obtiene el valor medio histórico de precipitación

La documentación completa de la API, con ejemplos de uso y esquemas de respuesta, está disponible en la interfaz Swagger en <http://localhost:8000/docs>.

4.3 Configuraciones a ajustar

Antes de realizar todo el despliegue se debe hacer la creación y modificación de algunos archivos para el correcto funcionamiento del aplicativo.

En primer lugar, dentro de `App/Deploy` se encuentra un archivo `env.example`, este debe ser copiado como se muestra a continuación (asumiendo que se está en sistemas UNIX):

```
1 cp env.example .env
```

Ahora, puede utilizar su editor de texto de preferencia y empezar a editar el archivo `.env`, asumiendo que posee el editor de texto GNU Emacs, se debe hacer:

```
1 emacs -nw .env
```

Posteriormente, deberá ajustar estas variables de acuerdo a sus requerimientos y disponibilidad de puertos para cada uno de los servicios mostrados.

Por otra parte, la contraseña de la base de datos deberá guardarla en un archivo de texto plano. Para eso, debe primero crear el directorio `Secrets` usando (desde el directorio raíz):

```
1 mkdir -p App/Secrets
```

Ya teniendo el directorio puede crear el archivo en donde estará la contraseña de la base de datos. Para esto, puede crear un archivo llamado `postgres_password` dentro de `Secrets/` de la siguiente manera:

```
1 cat > App/Secrets/postgres_password
2 Ingrese-su-password-segura
3 ^C
```

Donde `Ingrese-su-password-segura` es la contraseña de la base de datos y `^C` es la combinación de teclas Control + C. Así, ya tendrá las configuraciones de variables de ambiente para poder desplegar el aplicativo completo.

CAPÍTULO 5

Código y replicabilidad

Todo el código fuente se encuentra en el repositorio <https://github.com/GNUTADEO/Tuxilo/> en la rama `main`, ahí se encuentra todo el desarrollo, resultados, documentos y scripts necesarios para poder contribuir al proyecto bajo los términos de la licencia GPL-v3[10].

5.1 Frontend

El frontend de Tuxilo está desarrollado con SvelteKit, un framework moderno de JavaScript que proporciona una experiencia de desarrollo optimizada y un rendimiento excepcional en producción. La aplicación se encuentra en el directorio `App/Front/front/`.

5.1.1 Estructura del Frontend

Los archivos principales que se deben editar para modificar el frontend son:

- **`src/routes/+page.svelte`**: Página principal de la aplicación que contiene la lógica de selección de embalses, semestres y la consulta de datos proyectados.
- **`src/routes/+layout.svelte`**: Layout principal que define la estructura común de todas las páginas (navbar y footer).
- **`src/lib/components/Map.svelte`**: Componente del mapa interactivo usando Leaflet para visualizar embalses, estaciones de lluvia y caudal.
- **`src/lib/commons/Navbar.svelte`**: Componente de la barra de navegación superior.
- **`src/lib/commons/Footer.svelte`**: Componente del pie de página.

5.1.2 Configuración y Dependencias

El archivo `package.json` contiene todas las dependencias del proyecto. Las principales tecnologías utilizadas son:

- SvelteKit 2.x como framework principal

- Vite como bundler y servidor de desarrollo
- Tailwind CSS 4.x para estilos
- Leaflet para mapas interactivos
- DaisyUI como biblioteca de componentes

5.1.3 Scripts de Desarrollo

Para trabajar con el frontend, utilice los siguientes comandos desde el directorio `App/Front/front/`:

```
1 # Instalar dependencias
2 npm install
3
4 # Ejecutar servidor de desarrollo
5 npm run dev
6
7 # Compilar para produccion
8 npm run build
9
10 # Previsualizar build de produccion
11 npm run preview
12
13 # Verificar tipos y sintaxis
14 npm run check
```

5.1.4 Conexión con el Backend

El frontend se comunica con el backend a través de peticiones HTTP a la API REST. La URL base de la API es `http://localhost:8000/public/`. Los principales endpoints consultados son:

- `/embalses/points`: Obtiene las coordenadas de todos los embalses
- `/embalses/polygons`: Obtiene los polígonos de los embalses
- `/estaciones/flow`: Obtiene las estaciones de caudal
- `/estaciones/rain`: Obtiene las estaciones de precipitación
- `/data/q_proyectado/{embalse_id}/{periodo}`: Obtiene el caudal proyectado
- `/data/media/{graph_filename}`: Extrae valores de gráficos HTML

5.2 Backend

El backend de Tuxilo está desarrollado con FastAPI, un framework moderno de Python para construir APIs REST de alto rendimiento. La aplicación backend se encuentra en el directorio `App/Core/`.

5.2.1 Estructura del Backend

Los archivos principales del backend están organizados de la siguiente manera:

5.2.1.1 Archivo Principal

- **src/main.py:** Punto de entrada de la aplicación FastAPI. Define las APIs principales, configura CORS y monta los routers.

Ejemplo de código del archivo principal:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from routers.embalses import router as router_embalses
from routers.stations import router as router_stations
from routers.data import router as router_data

api_public = FastAPI(
    title="Endpoints públicos para Tuxhydro",
    version="0.1.0",
)

api_public.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

api_public.include_router(router_embalses)
api_public.include_router(router_stations)
api_public.include_router(router_data)
```

5.2.1.2 Routers (APIs)

Los routers definen los endpoints de la API y se encuentran en `src/routers/`:

- **routers/embalses.py**: Endpoints relacionados con embalses
 - GET /embalses/points: Retorna coordenadas de embalses como puntos
 - GET /embalses/polygons: Retorna geometrías de embalses como polígonos (GeoJSON)
- **routers/stations.py**: Endpoints relacionados con estaciones meteorológicas
 - GET /estaciones/flow: Lista todas las estaciones de caudal
 - GET /estaciones/rain: Lista todas las estaciones de precipitación
 - GET /estaciones/flow/nearest/{embalse_id}: Encuentra la estación de caudal más cercana a un embalse usando cálculos geoespaciales
- **routers/data.py**: Endpoints para acceder a datos históricos y proyectados
 - GET /data/semestres: Obtiene todos los periodos disponibles
 - GET /data/flow: Datos de caudal históricos
 - GET /data/rain: Datos de precipitación históricos
 - GET /data/q_proyectado/{embalse_id}/{periodo}: Calcula y retorna el caudal proyectado para un embalse y periodo específico
 - GET /data/media/{graph_filename}: Extrae valores estadísticos de archivos HTML de gráficos

Ejemplo de un endpoint con consulta geoespacial:

```

from geoalchemy2.functions import ST_Distance , ST_Transform
from sqlalchemy import select , cast
from geoalchemy2 import Geography

@router.get("/flow/nearest/{embalse_id}")
async def get_nearest_flow_station(
    embalse_id: int ,
    db: AsyncSession = Depends(get_db)
):
    """Obtiene la estación de caudal más cercana"""

    embalse_subq = select(ReservoirDot.geom).where(
        ReservoirDot.id == embalse_id
    ).scalar_subquery()

    distance_km = (ST_Distance(
        cast(ST_Transform(embalse_subq, 4326), Geography) ,
        cast(ST_SetSRID(

```

```

        ST_MakePoint(FlowStation.longitude,
                     FlowStation.latitude), 4326
    ), Geography)
) / 1000).label("distance_km")

stmt = select(
    FlowStation.id,
    FlowStation.name,
    distance_km
).order_by(distance_km).limit(1)

result = await db.execute(stmt)
return result.mappings().first()

```

5.2.1.3 Modelos de Base de Datos

Los modelos ORM de SQLAlchemy se encuentran en `src/models/` y definen la estructura de las tablas de la base de datos:

- **models/base.py**: Clase base para todos los modelos
- **models/reservoir_dot.py**: Modelo para embalses como puntos geográficos
- **models/reservoir_polygon.py**: Modelo para embalses como polígonos con geometría
- **models/flow_stations.py**: Modelo para estaciones de medición de caudal
- **models/rain_stations.py**: Modelo para estaciones de precipitación
- **models/flow.py**: Modelo para datos históricos de caudal
- **models/precipitation.py**: Modelo para datos históricos de precipitación
- **models/predicted.py**: Modelo para datos proyectados que incluye los coeficientes del modelo lluvia-escorrentía

Ejemplo del modelo de datos proyectados:

```

from sqlalchemy import String, BigInteger, Numeric
from sqlalchemy.orm import Mapped, mapped_column

class PredictedData(Base):
    __tablename__ = "predicted_data"
    __table_args__ = {"schema": "data"}

```

```
id: Mapped[int] = mapped_column(
    BigInteger, primary_key=True
)
embalse: Mapped[str] = mapped_column(String(50))
periodo: Mapped[str] = mapped_column(String(10))
precipitacion_project: Mapped[Decimal] = mapped_column(
    Numeric(25, 16)
)
ar: Mapped[Decimal] = mapped_column(Numeric(25, 16))
br: Mapped[Decimal] = mapped_column(Numeric(25, 4))
ar_km2: Mapped[Decimal] = mapped_column(Numeric(25, 3))
ai_km2: Mapped[Decimal] = mapped_column(Numeric(25, 3))
q_proyectado: Mapped[Decimal] = mapped_column(
    Numeric(25, 16)
)
```

5.2.2 Dependencias del Backend

El archivo `requirements.txt` en el directorio `App/` contiene todas las dependencias necesarias:

```
1 fastapi>=0.120.2
2 uvicorn>=0.38.0
3 sqlalchemy>=2.0.44
4 GeoAlchemy2==0.18.1
5 pandas>=2.3.3
6 aiofiles>=25.1.0
7 requests>=2.32.5
```

Las dependencias clave son:

- **FastAPI**: Framework web asíncrono
- **Uvicorn**: Servidor ASGI para ejecutar FastAPI
- **SQLAlchemy**: ORM para interactuar con PostgreSQL
- **GeoAlchemy2**: Extensión de SQLAlchemy para tipos geoespaciales (PostGIS)
- **Pandas**: Procesamiento y análisis de datos

5.2.3 Ejecutar el Backend

Para iniciar el servidor backend:

```
1 # Instalar dependencias
2 pip install -r requirements.txt
3
4 # Iniciar servidor de desarrollo
5 cd App/Core/src
6 uvicorn main:api --reload --port 8000
7
8 # El servidor estara disponible en:
9 # http://localhost:8000
10 # Documentacion interactiva en:
11 # http://localhost:8000/docs
```

5.2.4 Base de Datos

El backend requiere una base de datos PostgreSQL con la extensión PostGIS habilitada. La conexión se configura mediante variables de entorno o archivos de configuración. La base de datos contiene:

- Schema geodata: Tablas de estaciones y embalses con datos geoespaciales
- Schema data: Tablas de datos históricos y proyectados

5.3 Notebooks de análisis

El repositorio incluye diversos notebooks interactivos para análisis exploratorio y procesamiento de datos, ubicados en el directorio `notebooks/`. Estos notebooks permiten profundizar en los análisis realizados y generar nuevas visualizaciones.

5.3.1 Ejecución con Marimo

Los notebooks del proyecto están desarrollados con Marimo[25], un sistema de notebooks interactivos para Python que permite una ejecución reactiva y determinista. Para ejecutarlos, primero se debe instalar Marimo:

```
1 pip install marimo
```

Luego, se puede abrir cualquier notebook con:

```
1 marimo edit notebooks/Graficos.py
```

Cada notebook se opera desde el navegador de preferencia, un ejemplo lo puede ver en la figura 5.1

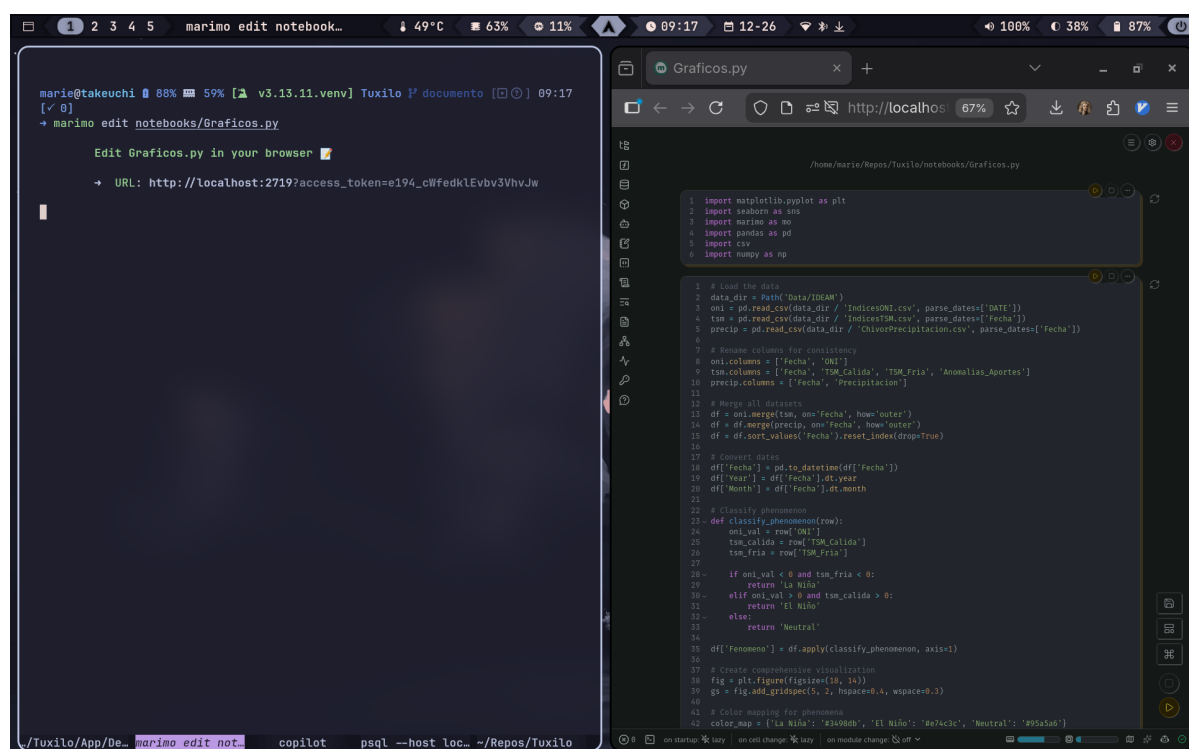


Figura 5.1: Ejecución y edición de un notebook de marimo

Los principales notebooks disponibles son:

- **Graficos.py**: Análisis de gráficos y visualizaciones generales
- **hidroelectrica.py**: Análisis específico del sector hidroeléctrico
- **fenomeno_presence.py**: Análisis de presencia de fenómenos climáticos (El Niño, La Niña)
- **precipitacion.py**: Análisis de datos de precipitación
- **CORRECTED_CAUDAL_CALCULATION.py**: Cálculo corregido de caudales proyectados
- **improve_br_cumulative.py**: Mejora de coeficientes de regresión lluvia-escorrentia
- **analisis_limpieza_caudal.py**: Limpieza y procesamiento de datos de caudal

5.4 Documentación

Toda la documentación está dentro de `docs/DocumentacionFinal/` y se puede modificar libremente bajo los términos de la licencia CC BY-SA 4.0[5].

5.4.1 Estructura de la Documentación

La documentación está organizada en los siguientes directorios:

- **chapters/**: Contiene todos los capítulos del documento (chapter_1.tex hasta chapter_7.tex)
- **appendices/**: Apéndices y acrónimos
- **frontmatter/**: Páginas preliminares como el resumen (abstract)
- **prefrontmatter/**: Portada, colofón y páginas anteriores al contenido principal
- **preamble/**: Configuraciones de LaTeX, fuentes y estilos
- **bibliography/**: Archivo de bibliografía en formato BibTeX
- **img/**: Imágenes y figuras utilizadas en el documento
- **graphics/**: Gráficos adicionales

5.4.2 Prerrequisitos para Compilar

Para compilar la documentación, se requieren las siguientes herramientas instaladas en el sistema:

- **XeLaTeX**: Compilador LaTeX con soporte para fuentes modernas
- **Biber**: Procesador de bibliografía para biblatex
- **Paquetes LaTeX**: memoir, biblatex, hyperref, graphicx, listings, y otros especificados en el preámbulo, se recomienda realizar la instalación de $\text{\LaTeX}$ con la máxima cantidad de paquetes
- **Make**: Herramienta para automatizar la compilación (opcional pero recomendado)

En sistemas Linux basados en Debian/Ubuntu, puede instalar todo lo necesario con:

```
1 sudo apt update
2 sudo apt install texlive-xetex texlive-bibtex-extra \
3 texlive-latex-extra texlive-fonts-extra biber make
```

En sistemas Arch Linux:

```
1 sudo pacman -S texlive-core texlive-latexextra \
2 texlive-bibtexextra biber make
```

5.4.3 Compilación de la Documentación

El proyecto incluye un Makefile que automatiza el proceso de compilación. Los comandos disponibles son:

```
1 # Compilacion completa (limpia y genera el PDF)
2 make all
3
4 # Compilacion incremental (mas rapida)
5 make
6
7 # Compilacion rapida de un solo paso
8 make quick
9
10 # Ver el PDF generado
11 make view
12
13 # Limpiar archivos auxiliares
14 make clean
15
16 # Limpiar todo incluyendo el PDF
17 make distclean
18
19 # Ver ayuda
20 make help
```

5.4.4 Proceso de Compilación Manual

Si prefiere compilar manualmente sin usar Make, ejecute los siguientes comandos en el directorio docs/DocumentacionFinal/:

```
1 # Primera pasada: procesa el documento y genera archivos auxiliares
2 xelatex -interaction=nonstopmode Report.tex
3
4 # Procesa la bibliografia
5 biber Report
6
7 # Segunda pasada: incorpora las referencias
8 xelatex -interaction=nonstopmode Report.tex
9
10 # Tercera pasada: resuelve referencias cruzadas
11 xelatex -interaction=nonstopmode Report.tex
```

El archivo PDF final se generará como Report.pdf.

5.4.5 Editar la Documentación

Para modificar el contenido de la documentación:

1. Edite los archivos `.tex` correspondientes en el directorio apropiado
2. Para agregar nuevas figuras, colóquelas en el directorio `img/`
3. Para agregar referencias bibliográficas, edite `bibliography/bibliography.bib`
4. Compile usando `make all` para regenerar el documento
5. Revise el PDF generado para verificar los cambios

CAPÍTULO 6

Resultados y visualización

CAPÍTULO 7

Guía de despliegue e instalación operativa

Este capítulo presenta una guía detallada para la instalación, configuración y despliegue operativo del sistema de análisis hidroeléctrico y climático Tuxhydro. Se describen los requerimientos técnicos mínimos, los procedimientos de instalación en diferentes entornos, las instrucciones para ejecutar e interpretar la solución, así como recomendaciones para su uso institucional y posibles escenarios de escalabilidad.

7.1 Requerimientos técnicos

Para garantizar el correcto funcionamiento del sistema, se requiere contar con un entorno que cumpla con las especificaciones mínimas detalladas a continuación. La solución está diseñada para ser independiente del sistema operativo, gracias al uso de contenedores Docker, lo que facilita su despliegue en diferentes plataformas.

7.1.1 Requerimientos de hardware

Los requerimientos mínimos de hardware para el despliegue del sistema completo son los siguientes:

- **Procesador:** CPU de 4 núcleos (x86_64)
- **Memoria RAM:** Mínimo 8 GB, recomendado 16 GB para entornos de producción
- **Almacenamiento:** Mínimo 20 GB de espacio disponible en disco
- **Conectividad:** Acceso a Internet para la descarga de dependencias e imágenes de contenedores (solo durante la instalación inicial)

Para entornos de desarrollo o análisis exploratorio con notebooks, se recomiendan especificaciones más modestas (2 núcleos, 4 GB RAM), ya que no es necesario ejecutar todos los servicios simultáneamente.

7.1.2 Requerimientos de software

El sistema requiere la instalación previa de las siguientes herramientas:

- **Docker:** Versión 20.10 o superior[29]
- **Docker Compose:** Versión 2.0 o superior (incluido en Docker Desktop)
- **Git:** Versión 2.30 o superior para la clonación del repositorio[15]
- **Navegador web moderno:** Chrome, Firefox, Edge o Safari (versiones recientes con soporte para ES6+)

Para el desarrollo y ejecución de notebooks de análisis de datos se requiere adicionalmente:

- **Python:** Versión 3.11 o superior[37]
- **Node.js:** Versión 18 o superior (opcional, solo para desarrollo del frontend)
- **Bun:** Versión 1.0 o superior (alternativa a Node.js para mejor rendimiento)[34]

7.1.3 Sistemas operativos soportados

La solución ha sido probada y es compatible con los siguientes sistemas operativos:

- **Linux:** Distribuciones basadas en Debian (Debian 12+), Arch Linux y NixOS.

Por el momento no se ha probado en más sistemas operativos, se invita al lector a que realice pruebas en diferentes sistemas y contribuir al proyecto.

Se recomienda el uso de sistemas operativos basados en Linux para entornos de producción, debido a su mejor rendimiento y estabilidad en la ejecución de contenedores Docker.

7.2 Instalación y configuración

Esta sección describe el proceso completo de instalación del sistema, desde la obtención del código fuente hasta la configuración inicial de los servicios.

7.2.1 Obtención del código fuente

El código fuente del proyecto se encuentra alojado en el repositorio público de GitHub. Para obtenerlo, se debe ejecutar el siguiente comando en la terminal:

```
1 git clone https://github.com/GNUTADEO/Tuxilo.git
2 cd Tuxilo
```

Una vez clonado el repositorio, se recomienda verificar la estructura de directorios principal, que debe ser similar a la presentada en el README del proyecto (ver `README.md`).

7.2.2 Configuración del entorno

Antes de iniciar los servicios, es necesario configurar las variables de entorno que utilizará el sistema. El repositorio incluye un archivo de ejemplo de configuración ubicado en `App/Deploy/env.example`. Para configurar el sistema, se debe crear un archivo `.env` a partir de este ejemplo:

```
1 cd App/Deploy
2 cp env.example .env
```

El archivo `.env` contiene las siguientes variables de configuración principal:

- `PORT_FRONT`: Puerto para el frontend (por defecto 1580)
- `PORT_CORE`: Puerto para la API del backend (por defecto 8000)
- `PORT_DB_PG`: Puerto de la base de datos PostgreSQL (por defecto 5432)
- `PORT_PGADMIN`: Puerto para la interfaz de administración pgAdmin (por defecto 8081)
- `POSTGRES_USER`: Usuario de la base de datos (por defecto postgres)
- `POSTGRES_DB`: Nombre de la base de datos (por defecto tuxhydro)
- `PGADMIN_DEFAULT_EMAIL`: Correo de acceso a pgAdmin
- `PGADMIN_DEFAULT_PASSWORD`: Contraseña de acceso a pgAdmin

Importante: En entornos de producción, se debe modificar obligatoriamente la contraseña de la base de datos y las credenciales de pgAdmin por valores seguros.

7.2.3 Configuración de secretos

La contraseña de la base de datos se gestiona mediante Docker Secrets para mayor seguridad. Es necesario crear el archivo de secreto antes del despliegue:

```
1 mkdir -p App/Secrets
2 echo "tu_contraseña_segura" > App/Secrets/postgres_password
3 chmod 600 App/Secrets/postgres_password
```

Este archivo debe contener únicamente la contraseña en texto plano, sin saltos de línea adicionales.

7.2.4 Construcción de las imágenes

El sistema utiliza múltiples contenedores Docker orquestados mediante Docker Compose. Para construir todas las imágenes necesarias, se proporciona un script de construcción automatizado:

```
1 cd App/Deploy
2 docker compose build
```

Construye las siguientes imágenes:

- **tuxhydro_api_core**: Servidor de API FastAPI con la lógica de negocio
- **tuxhydro_frontend**: Aplicación web frontend construida con SvelteKit
- **tuxhydro_frontend_dev**: Versión de desarrollo del frontend con hot-reload

El proceso de construcción puede tomar entre 5 y 15 minutos dependiendo de la velocidad de la conexión a Internet y los recursos del sistema y deberá obtener una salida similar a la mostrada en la figura 7.1.


```
+ docker compose build
[+] Building 0.6s (40/40) FINISHED
  => [internal] load local bake definitions                                0.0s
  => reading from stdin 1.44kB                                           0.0s
  => [api_core internal] load build definition from Dockerfile            0.0s
  => transferring dockerfile: 1.35kB                                      0.0s
  => [frontend_dev internal] load build definition from Dockerfile        0.0s
  => transferring dockerfile: 287B                                         0.0s
  => [frontend internal] load build definition from Dockerfile           0.1s
  => transferring dockerfile: 616B                                         0.0s
  => [api_core internal] load metadata for docker.io/library/python:3.14-slim-bookworm 0.0s
  => [api_core internal] load .dockerignore                                0.0s
  => transferring context: 2B                                               0.0s
  => [frontend internal] load metadata for docker.io/oven/bun:1           0.0s
  => [frontend internal] load .dockerignore                                0.0s
  => transferring context: 2B                                               0.0s
  => [api_core 1/13] FROM docker.io/library/python:3.14-slim-bookworm     0.0s
  => [api_core internal] load build context                               0.0s
  => transferring context: 883B                                             0.0s
  => [frontend_dev 1/5] FROM docker.io/oven/bun:1                         0.0s
  => [frontend internal] load build context                               0.0s
  => transferring context: 2.49kB                                           0.0s
  => CACHED [api_core 2/13] WORKDIR /api                                   0.0s
  => CACHED [api_core 3/13] RUN apt-get update && apt-get install -y gcc g++ libpq-dev curl jq && rm -rf /var/lib/apt/lists/* 0.0s
  => CACHED [api_core 4/13] COPY src/packages/shared_db.zip ./shared_db.zip 0.0s
  => CACHED [api_core 5/13] COPY src/packages/models_stub.zip ./models_stub.zip 0.0s
  => CACHED [api_core 6/13] RUN pip install --no-cache-dir ./shared_db.zip && pip install --no-cache-dir ./models_stub.zip && rm *.zip 0.0s
  => CACHED [api_core 7/13] COPY src/pyproject.toml ./pyproject.toml      0.0s
  => CACHED [api_core 8/13] RUN pip install --no-cache-dir -e .            0.0s
  => CACHED [api_core 9/13] COPY src/main.py ./main.py                    0.0s
  => CACHED [api_core 10/13] COPY src/routers ./routers                    0.0s
  => CACHED [api_core 11/13] COPY src/models ./models                      0.0s
  => CACHED [api_core 12/13] COPY entrypoint_core.sh ./entrypoint_core.sh 0.0s
  => CACHED [api_core 13/13] RUN chmod +x ./entrypoint_core.sh            0.0s
  => CACHED [frontend 2/5] WORKDIR /app                                    0.0s
  => CACHED [frontend 3/5] COPY ../front/package.json ../front/bun.lock ./ 0.0s
  => CACHED [frontend_dev 4/5] RUN bun install                             0.0s
  => CACHED [frontend_dev 5/5] COPY ../front/ ./                           0.0s
  => CACHED [frontend build 4/6] RUN bun install --frozen-lockfile         0.0s
  => CACHED [frontend build 5/6] COPY ../front/ ./                         0.0s
  => CACHED [frontend build 6/6] RUN bun run build                         0.0s
  => CACHED [frontend runtime 3/5] COPY --from=build /app/build ./build    0.0s
  => CACHED [frontend runtime 4/5] COPY --from=build /app/node_modules ./node_modules 0.0s
  => CACHED [frontend runtime 5/5] COPY --from=build /app/package.json ./ 0.0s
  => [api_core] exporting to image                                         0.0s
  => exporting layers                                                       0.0s
```

Figura 7.1: Proceso terminado del build

7.3 Despliegue y ejecución

Una vez completada la instalación y configuración, el sistema está listo para su despliegue. Esta sección describe cómo iniciar, detener y gestionar los diferentes servicios.

7.3.1 Inicio de servicios

Para iniciar todos los servicios del sistema en modo producción, se debe ejecutar:

```
1 cd App/Deploy
2 docker compose up -d
```

El flag `-d` indica que los contenedores se ejecutarán en segundo plano (modo *detached*). Docker Compose iniciará los siguientes servicios en el orden adecuado:

1. **db (gis_db)**: Base de datos PostgreSQL con extensión PostGIS[36]
2. **api_core**: Servidor de API que expone los endpoints de datos
3. **frontend**: Aplicación web de visualización

4. **frontend_dev**: Servidor de desarrollo (opcional)
5. **nginx**: Servidor web reverse proxy
6. **pgadmin**: Interfaz web de administración de base de datos

En la figura 7.2 se muestra cómo deberían estar los servicios y la salida del `docker compose up -d`.

```
+ docker compose up -d
WARN[0000] a network with name net_gis_tuxhydro exists but was not created for project "deploy".
Set 'external: true' to use an existing network
WARN[0000] a network with name net_nginx_tuxhydro exists but was not created for project "deploy".
Set 'external: true' to use an existing network
WARN[0000] a network with name net_api_tuxhydro exists but was not created for project "deploy".
Set 'external: true' to use an existing network
[+] up 8/8
✓ Volume gis_vol_pg          Created      0.1s
✓ Volume pgadmin_vol_pgadmin Created      0.0s
✓ Container tuxhydro_frontend Created      0.1s
✓ Container gis_db           Healthy     11.0s
✓ Container tuxhydro_frontend_dev Created      0.1s
✓ Container pgadmin_web       Created      0.1s
✓ Container tuxhydro_api_core Created      0.0s
✓ Container tuxhydro_nginx    Created      0.0s

marie@takeuchi ~ 93% 59% Tuxilo/App/Deploy P documento [ ] 89:08 [✓ 0]
+ docker ps
Found existing alias for "docker ps". You should use: "dps"
CONTAINER ID   IMAGE          COMMAND                  CREATED     STATUS              PORTS                               NAMES
cc66a2b4d49f   nginx:latest   "/docker-entrypoint..." 50 seconds ago Up 39 seconds       0.0.0.0:5173→5173/tcp, [::]:5173→5173/tcp, 0.0.0.0:8000→8000/tcp, [::]:8000→8000/tcp, 80/tcp, 0.0.0.0:8081→8081/tcp, [::]:8081→8081/tcp, 0.0.0.0:1580→3000/tcp, [::]:1580→3000/tcp tuxhydro_nginx
f40094eadb2    tuxhydro_api_core "/entrypoint_core.sh..." 50 seconds ago Up 39 seconds                               tuxhydro_api_core
3ff00464cbb5   postgis/postgis "docker-entrypoint.s..." 51 seconds ago Up 50 seconds (healthy) 0.0.0.0:5432→5432/tcp, [::]:5432→5432/tcp gis_db
80667fe9df0a   dpagadmin4:9.10 "/entrypoint.sh"         51 seconds ago Up 50 seconds       80/tcp, 443/tcp                pgadmin_web
4990ad60b042   tuxhydro_frontend_dev "/usr/local/bin/dock..." 51 seconds ago Up 50 seconds                               tuxhydro_frontend_dev
97d7665e3b23   tuxhydro_frontend "/usr/local/bin/dock..." 51 seconds ago Up 50 seconds       3000/tcp                       tuxhydro_frontend
```

Figura 7.2: Servicios corriendo en la maquina local

La primera vez que se ejecuta el sistema, la base de datos se inicializa automáticamente ejecutando los scripts SQL ubicados en `scripts/ddl/` y `scripts/dml/`, que crean las tablas necesarias y cargan los datos iniciales desde los archivos CSV ubicados en `data/`.

7.3.2 Verificación del despliegue

Para verificar que todos los contenedores se han iniciado correctamente, se puede ejecutar:

```
1 docker compose ps
```

Este comando debe mostrar todos los servicios en estado `running` (figura 7.2). En caso de que algún servicio presente errores, se pueden consultar los logs con:

```
1 docker compose logs nombre_servicio
```

Por ejemplo, para ver los logs del API:

```
1 docker compose logs api_core
```

7.3.3 Acceso a la aplicación

Una vez que todos los servicios estén operativos, se puede acceder a las diferentes interfaces del sistema a través de las siguientes URLs (considerando la configuración por defecto):

- **Aplicación web principal:** `http://localhost:1580`
- **Versión de desarrollo:** `http://localhost:5173` (con hot-reload)
- **API REST:** `http://localhost:8000`
- **Documentación API (Swagger):** `http://localhost:8000/docs`
- **pgAdmin:** `http://localhost:8081`

El acceso a pgAdmin requiere las credenciales configuradas en el archivo `.env`. Una vez autenticado, se puede conectar al servidor de base de datos utilizando el hostname `gis_db` (nombre del contenedor dentro de la red Docker).

7.3.4 Detención y reinicio de servicios

Para detener todos los servicios preservando los datos:

```
1 docker compose stop
```

Para reiniciar los servicios:

```
1 docker compose start
```

Para detener y eliminar los contenedores (manteniendo los volúmenes de datos):

```
1 docker compose down
```

Para eliminar completamente todo, incluyendo volúmenes de datos:

```
1 docker compose down -v
```

Advertencia: El último comando eliminará toda la información de la base de datos. Usar solo cuando se desee reiniciar completamente el sistema.

7.4 Recomendaciones para uso institucional

Para una implementación exitosa del sistema en un entorno institucional, se presentan las siguientes recomendaciones organizadas por área.

7.4.1 Seguridad y acceso

- **Autenticación:** Para ambientes de producción, se recomienda implementar un sistema de autenticación y autorización en la API, utilizando tokens JWT o OAuth2
- **HTTPS:** Configurar certificados SSL/TLS para acceso seguro mediante HTTPS, utilizando Let's Encrypt o certificados institucionales
- **Firewall:** Limitar el acceso a los puertos de servicio únicamente desde redes autorizadas
- **Secretos:** Nunca almacenar contraseñas en el control de versiones. Utilizar sistemas de gestión de secretos como HashiCorp Vault o AWS Secrets Manager
- **Actualizaciones:** Mantener actualizadas las imágenes de Docker y las dependencias para corregir vulnerabilidades de seguridad

7.4.2 Monitoreo y mantenimiento

- **Logs centralizados:** Implementar un sistema de agregación de logs como ELK Stack (Elasticsearch, Logstash, Kibana) o Grafana Loki
- **Métricas:** Configurar Prometheus para recolectar métricas de los contenedores y servicios
- **Alertas:** Establecer alertas automáticas para condiciones críticas (caída de servicios, uso excesivo de recursos, errores en la API)
- **Respaldos:** Programar respaldos automáticos de la base de datos PostgreSQL utilizando `pg_dump` o soluciones de backup continuo
- **Documentación operativa:** Mantener documentación actualizada de procedimientos operativos, contactos de soporte y planes de contingencia

7.4.3 Actualización de datos

El sistema requiere actualización periódica de los datos para mantener la precisión de las proyecciones:

- **Datos climáticos:** Actualizar mensualmente los índices oceánicos desde las fuentes oficiales (NOAA, CPC)
- **Datos hidrológicos:** Integrar nuevos registros de caudal y precipitación del IDEAM[19] y del GRDC[17]
- **Recalibración:** Revisar y recalibrar semestralmente los coeficientes de regresión (a_r , b_r) con los nuevos datos disponibles

- **Validación:** Comparar las proyecciones realizadas con las observaciones reales para evaluar la precisión del modelo

Para automatizar la actualización de datos, se puede desarrollar un pipeline ETL (Extract, Transform, Load) utilizando herramientas como Apache Airflow o prefect.

7.5 Escalabilidad y extensibilidad

El sistema ha sido diseñado con arquitectura modular que facilita su escalabilidad horizontal y vertical, así como su extensión funcional.

7.5.1 Escalabilidad horizontal

Para soportar mayor carga de usuarios concurrentes, se puede escalar horizontalmente mediante las siguientes estrategias:

- **Múltiples instancias de API:** Utilizar Docker Swarm o Kubernetes para desplegar múltiples réplicas del servicio `api_core`
- **Balanceo de carga:** Configurar NGINX o HAProxy como load balancer para distribuir las peticiones entre las instancias
- **Caché distribuida:** Implementar Redis o Memcached para cachear respuestas frecuentes y reducir la carga en la base de datos
- **CDN:** Utilizar una red de distribución de contenidos (CloudFlare, AWS CloudFront) para servir los archivos estáticos del frontend

7.5.2 Escalabilidad vertical

Para manejar consultas más complejas o datasets más grandes:

- **Base de datos:** Incrementar los recursos de la base de datos PostgreSQL (CPU, RAM, SSD) y optimizar las consultas con índices adicionales
- **Particionamiento:** Implementar particionamiento de tablas por fechas para mejorar el rendimiento de consultas temporales
- **Read replicas:** Configurar réplicas de lectura de PostgreSQL para distribuir las consultas de solo lectura

7.5.3 Extensiones funcionales

El sistema puede extenderse para incorporar nuevas funcionalidades:

- **Nuevos índices climáticos:** Integrar índices adicionales como PDO (Pacific Decadal Oscillation), NAO (North Atlantic Oscillation), integrando nuevas columnas en la base de datos y actualizando los modelos predictivos
- **Análisis de series temporales avanzado:** Implementar modelos ARIMA, Prophet o redes neuronales recurrentes (LSTM) para mejorar las proyecciones
- **Alertas automáticas:** Desarrollar un sistema de notificaciones que envíe alertas por correo o SMS cuando se detecten escenarios críticos
- **Dashboard ejecutivo:** Crear visualizaciones adicionales con indicadores clave de desempeño (KPIs) para toma de decisiones gerenciales
- **Integración con otros sistemas:** Conectar con sistemas de generación eléctrica para estimar impactos en la producción de energía
- **Módulo de optimización:** Desarrollar algoritmos de optimización para la gestión óptima de embalses considerando las proyecciones

7.5.4 Migración a infraestructura en la nube

Para organizaciones que prefieran infraestructura en la nube, el sistema puede desplegarse en:

- **AWS:** Utilizando ECS (Elastic Container Service) para los contenedores, RDS para PostgreSQL con PostGIS, y S3 para almacenamiento de datos
- **Google Cloud Platform:** Con Google Kubernetes Engine (GKE), Cloud SQL, y Cloud Storage
- **Azure:** Utilizando Azure Kubernetes Service (AKS), Azure Database for PostgreSQL, y Azure Blob Storage

En todos los casos, se recomienda utilizar servicios administrados de base de datos para reducir la carga operativa y aprovechar características como backups automáticos, alta disponibilidad y escalabilidad gestionada.

7.6 Solución de problemas comunes

Esta sección presenta soluciones a problemas frecuentes durante la instalación y operación del sistema.

7.6.1 Error al iniciar la base de datos

Síntoma: El contenedor `gis_db` no inicia o reinicia constantemente.

Posibles causas y soluciones:

- Verificar que el archivo de secreto existe: `ls -la App/Secrets/postgres_password`
- Revisar los logs del contenedor: `docker compose logs db`
- Verificar que el puerto 5432 no esté ocupado: `netstat -tuln | grep 5432`
- Eliminar el volumen y reiniciar: `docker compose down -v && docker compose up -d`

7.6.2 Frontend no carga correctamente

Síntoma: La página web no se muestra o aparecen errores en la consola del navegador.

Posibles causas y soluciones:

- Verificar que el API está respondiendo: `curl http://localhost:8000/docs`
- Limpiar la caché del navegador (Ctrl+Shift+Del)
- Revisar logs del frontend: `docker compose logs frontend`
- Reconstruir la imagen del frontend: `docker compose build frontend --no-cache`

7.6.3 Datos no se cargan en la base de datos

Síntoma: La aplicación funciona pero no muestra embalses o datos de caudal.

Posibles causas y soluciones:

- Verificar que los archivos CSV existen en los directorios esperados (`data/Caudal/`, `data/Precipitación/`)
- Conectar a la base de datos con pgAdmin y verificar que las tablas contienen datos
- Revisar logs del proceso de inicialización: `docker compose logs db | grep -i error`
- Ejecutar manualmente los scripts SQL: conectarse a la base de datos y ejecutar los scripts en orden desde `scripts/ddl/` y `scripts/dml/`

7.6.4 Problemas de rendimiento

Síntoma: La aplicación responde lentamente o las consultas tardan mucho.

Posibles causas y soluciones:

- Incrementar recursos asignados a Docker (Docker Desktop → Settings → Resources)
- Crear índices en la base de datos para las columnas más consultadas
- Implementar caché en la API para respuestas frecuentes
- Optimizar consultas SQL revisando el plan de ejecución con `EXPLAIN ANALYZE`

Bibliografía

- [1] Marcelo Trylesinski (Kludex). *Kludex/uvicorn: An ASGI web server for Python*. <https://github.com/Kludex/uvicorn>. Accessed: 2025-12-25. GitHub, 2025.
- [2] *About OpenStreetMap*. Official OpenStreetMap “About” page. OpenStreetMap Foundation. 2025. URL: <https://www.openstreetmap.org/about> (visitado 14 de diciembre de 2025).
- [3] Diego Alonso. *Qué es y qué podemos hacer con GRASS GIS*. Accedido: diciembre 26, 2025. MappingGIS. 2016. URL: <https://mappinggis.com/2016/05/puedo-grass-gis-7/>.
- [4] *Calc: The Spreadsheet for Everyone*. Official LibreOffice Calc product page. The Document Foundation. 2025. URL: <https://www.libreoffice.org/discover/calc/> (visitado 14 de diciembre de 2025).
- [5] Creative Commons. *Creative Commons Attribution -ShareAlike 4.0 International (CC BY-SA 4.0) License*. <https://creativecommons.org/licenses/by-sa/4.0/>. Accessed: 2025-12-25. Creative Commons, 2025.
- [6] drawio Contributors. *LICENSE – jgraph/drawio-desktop*. Accessed: 2025-12-23. GitHub. 2025. URL: <https://github.com/jgraph/drawio-desktop/blob/dev/LICENSE>.
- [7] Equipo Analítica XM. *API_XM: API para acceso a información del mercado eléctrico XM*. https://github.com/EquipoAnaliticaXM/API_XM. Accessed: 2025-12-25. GitHub, 2025.
- [8] FastAPI Contributors. *LICENSE – fastapi/fastapi*. Accessed: 2025-12-23. GitHub. 2025. URL: <https://github.com/fastapi/fastapi/blob/master/LICENSE>.
- [9] Python Software Foundation. *psf/requests: A simple, yet elegant, HTTP library for Python*. <https://github.com/psf/requests>. Accessed: 2025-12-25. GitHub, 2025.
- [10] Free Software Foundation. *GNU General Public License, version 3.0*. <https://www.gnu.org/licenses/gpl-3.0.en.html>. Accessed: 2025-12-25. GNU Project, 2007.

- [11] Free Software Foundation. *GNU Make Manual*. Accessed: 2025-12-23. Free Software Foundation. 2025. URL: <https://www.gnu.org/software/make/manual/make.html>.
- [12] Free Software Foundation. *What is Free Software?* Accessed: 2025-12-23. Free Software Foundation. 2025. URL: <https://www.gnu.org/philosophy/free-sw.html>.
- [13] Eric Gazoni y Charlie Clark. *openpyxl: A Python library to read/write Excel 2010 xlsx/xlsm files*. <https://github.com/ericgazoni/openpyxl>. Accessed: 2025-12-25. GitHub, 2025.
- [14] GeoAlchemy2 Development Team. *geoalchemy/geoalchemy2: Extensions for SQLAlchemy to support spatial databases*. <https://github.com/geoalchemy/geoalchemy2>. Accessed: 2025-12-25. GitHub, 2025.
- [15] Git SCM. *About Git*. Accessed: 2025-12-23. Git SCM. 2025. URL: <https://git-scm.com/about>.
- [16] Gitea Contributors. *About Gitea*. Accessed: 2025-12-23. Gitea. 2025. URL: <https://about.gitea.com/>.
- [17] Global Runoff Data Centre. *GRDC Data Portal*. Accessed: 2025-12-23. Global Runoff Data Centre, Federal Institute of Hydrology (BfG), Koblenz, Germany. 2025. URL: <https://portal.grdc.bafg.de/applications/public.html?publicuser=PublicUser#dataDownload/Subregions>.
- [18] GNUTADEO. *Tuxilo: Análisis Hidroeléctrico y Climático*. GitHub repository. 2025. URL: <https://github.com/GNUTADEO/Tuxilo/> (visitado 10 de diciembre de 2025).
- [19] Instituto de Hidrología, Meteorología y Estudios Ambientales (IDEAM). *DHIME – Sistema de Información para la gestión de datos Hidrológicos y Meteorológicos*. Accedido: diciembre 26, 2025. 2025. URL: <http://dhime.ideam.gov.co/atencionciudadano/>.
- [20] Instituto de Hidrología, Meteorología y Estudios Ambientales (IDEAM). *Zonificación Hidrográfica*. Consulta: diciembre 25, 2025. IDEAM. 2025. URL: <http://www.ideam.gov.co/web/agua/zonificacion-hidrografica>.
- [21] Instituto Geográfico Agustín Codazzi (IGAC). *Portal de información geo-espacial – Colombia en Mapas*. Accedido: 20 de diciembre de 2025. IGAC. 2025. URL: <https://www.colombiaenmapas.gov.co/?u=0&t=39&servicio=1468> (visitado 20 de diciembre de 2025).
- [22] LaTeX Project. *LaTeX Project Public License (LPPL)*. Accessed: 2025-12-23. LaTeX Project. 2025. URL: <https://www.latex-project.org/lppl/>.
- [23] Leaflet Contributors. *LICENSE – Leaflet/Leaflet*. Accessed: 2025-12-23. GitHub. 2025. URL: <https://github.com/Leaflet/Leaflet/blob/main/LICENSE>.

- [24] Leaflet contributors. *Leaflet — a JavaScript library for interactive maps*. Accedido el 16 de diciembre de 2025. Leaflet. 2025. URL: <https://leafletjs.com/> (visitado 16 de diciembre de 2025).
- [25] MARIMO Contributors. *LICENSE – marimo/marimo*. Accessed: 2025-12-23. GitHub. 2025. URL: <https://github.com/marimo-team/marimo/blob/main/LICENSE>.
- [26] Matplotlib Development Team. *Matplotlib: License information and terms*. <https://matplotlib.org/stable/project/license.html>. Accessed: 2025-12-25. Matplotlib, 2025.
- [27] Microsoft. *microsoft/TypeScript: TypeScript programming language*. <https://github.com/microsoft/TypeScript>. Accessed: 2025-12-25. GitHub, 2025.
- [28] Ministerio de Minas y Energía de Colombia. *Colombia apuesta a la aplicación de un estándar mundial de sostenibilidad en energía hidroeléctrica*. Accedido: diciembre 26, 2025. 2023. URL: <https://www.minenergia.gov.co/es/sala-de-prensa/noticias-index/colombia-como-uno-de-los-1%C3%ADderes-latinoamericanos-en-energ%C3%ADa-hidroel%C3%A9ctrica-le-apuesta-a-la-aplicaci%C3%B3n-de-un-est%C3%A1ndar-mundial-de-sostenibilidad-con-el-apoyo-de-la-cooperaci%C3%B3n-econ%C3%B3mica-y-desarrollo-seco-de-suiza-y-la-asociaci%C3%B3n-internacional-de-energ%C3%AD/>.
- [29] Moby Contributors. *LICENSE – moby/moby*. Accessed: 2025-12-23. GitHub. 2025. URL: <https://github.com/moby/moby/blob/master/LICENSE>.
- [30] NGINX, Inc. *NGINX License*. Accessed: 2025-12-23. NGINX, Inc. 2025. URL: <https://nginx.org/LICENSE>.
- [31] NumPy Developers. *NumPy: The fundamental package for scientific computing with Python*. <https://numpy.org/>. Accessed: 2025-12-25. NumPy, 2025.
- [32] *Open Data Commons Open Database License (ODbL)*. Licencia de base de datos abierta con condiciones de atribución y “share-alike” para datos y bases de datos. Open Data Commons / Open Knowledge Foundation. 2025. URL: <https://opendatacommons.org/licenses/odbl/> (visitado 16 de diciembre de 2025).
- [33] OpenTopography. *Programmatic Access to OpenTopography’s Point Cloud Data with Tile Indexes*. Consulta: diciembre 25, 2025. 2025. URL: <https://opentopography.org/blog/programmatic-access-opentopographys-point-cloud-data-tile-indexes>.
- [34] oven-sh. *oven-sh/bun: An incredibly fast JavaScript runtime, bundler, test runner, and package manager*. <https://github.com/oven-sh/bun>. Accessed: 2025-12-25. GitHub, 2025.

- [35] Plotly Development Team. *plotly/plotly.py: An interactive graphing library for Python*. <https://github.com/plotly/plotly.py>. Accessed: 2025-12-25. GitHub, 2025.
- [36] PostGIS Project Steering Committee. *PostGIS: Spatial and Geographic Objects for PostgreSQL*. Accessed: 2025-12-14. PostGIS Project. 2025. URL: <https://postgis.net/>.
- [37] Python Software Foundation. *History and License — Python 3 Documentation*. Accessed: 2025-12-23. Python Software Foundation. 2025. URL: <https://docs.python.org/3/license.html>.
- [38] QGIS Development Team. *QGIS Licensing*. Accessed: 2025-12-23. QGIS Project. 2025. URL: <https://qgis.org/license/>.
- [39] Saadeghi. *saadeghi/daisyui: A Tailwind CSS component library*. <https://github.com/saadeghi/daisyui>. Accessed: 2025-12-25. GitHub, 2025.
- [40] Seaborn Development Team. *mwaskom/seaborn: Statistical data visualization library for Python*. <https://github.com/mwaskom/seaborn>. Accessed: 2025-12-25. GitHub, 2025.
- [41] Jan Sitterson et al. «An Overview of Rainfall-Runoff Model Types». En: *Proceedings of the 9th International Congress on Environmental Modelling and Software (iEMSs 2018): Modelling for Sustainable Food–Energy–Water Systems*. Accedido: diciembre 26, 2025. 2018. URL: <https://scholarsarchive.byu.edu/iemssconference/>.
- [42] SQLAlchemy Development Team. *sqlalchemy/sqlalchemy: The Database Toolkit and Object Relational Mapper for Python*. <https://github.com/sqlalchemy/sqlalchemy>. Accessed: 2025-12-25. GitHub, 2025.
- [43] Svelte Contributors. *LICENSE.md – sveltejs/svelte*. Accessed: 2025-12-23. GitHub. 2025. URL: <https://github.com/sveltejs/svelte/blob/main/LICENSE.md>.
- [44] Tailwind Labs. *tailwindlabs/tailwindcss: A utility-first CSS framework*. <https://github.com/tailwindlabs/tailwindcss>. Accessed: 2025-12-25. GitHub, 2025.
- [45] The pandas development team. *pandas-dev/pandas: Flexible and powerful data analysis / manipulation library for Python*. Versión 2.3.0. Accessed: 2025-12-25. 2025. DOI: 10.5281/zenodo.15597513. URL: <https://doi.org/10.5281/zenodo.15597513>.
- [46] Tin Tvrtković. *aiofiles: File support for asyncio*. <https://github.com/Tinche/aiofiles>. Accessed: 2025-12-25. GitHub, 2025.
- [47] Vite Development Team. *vitejs/vite: A next-generation frontend tooling build tool*. <https://github.com/vitejs/vite>. Accessed: 2025-12-25. GitHub, 2025.

- [48] Wikipedia contributors. *Bash (Unix shell)*. Accessed: 2025-12-23. Wikimedia Foundation. 2025. URL: [https://en.wikipedia.org/wiki/Bash_\(Unix_shell\)](https://en.wikipedia.org/wiki/Bash_(Unix_shell)).
- [49] Wikipedia contributors. *GeoJSON*. Accedido el 16 de diciembre de 2025. Wikipedia. 2025. URL: <https://en.wikipedia.org/wiki/GeoJSON> (visitado 16 de diciembre de 2025).
- [50] Wikipedia contributors. *GitHub*. Accessed: 2025-12-23. Wikimedia Foundation. 2025. URL: <https://en.wikipedia.org/wiki/GitHub>.
- [51] Wikipedia contributors. *GNU Emacs*. Accessed: 2025-12-23. Wikimedia Foundation. 2025. URL: https://en.wikipedia.org/wiki/GNU_Emacs.